SINGLE LINKED LIST

1.insertion at beginning

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertBegin(struct Node *head, int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    newnode->next = head;
    head = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
```

```
10 20 30

=== Code Execution Successful ===
```

main.c Share Run Output

```c
struct Node* insertBegin(struct Node *head, int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    newnode->next = head;
    head = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *head = NULL;
    head = insertBegin(head,30);
    head = insertBegin(head,20);
    head = insertBegin(head,10);
    display(head);
}
```

```
10 20 30

=== Code Execution Successful ===
```

2.insertion at end

Programiz
C Online Compiler

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertEnd(struct Node *head,int data)
{
    struct Node *newnode = (struct Node*)malloc(sizeof(struct Node
        ));
    newnode->data = data;
    newnode->next = NULL;
    if(head == NULL)
        return newnode;
    struct Node *temp = head;

    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
```

Output

```
5 10 15

=== Code Execution Successful ===
```

SBI LIFE INSURANCE CO. Learn More

Programiz
C Online Compiler

```c
        return newnode;
    struct Node *temp = head;

    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *head = NULL;
    head = insertEnd(head,5);
    head = insertEnd(head,10);
    head = insertEnd(head,15);
    display(head);
}
```

Output

```
5 10 15

=== Code Execution Successful ===
```

SBI LIFE INSURANCE CO. Learn More

3.insertion at given position

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertPos(struct Node *head,int data,int pos)
{
    struct Node *newnode = malloc(sizeof(struct Node));
    newnode->data = data;

    if(pos==1)
    {
        newnode->next = head;
        return newnode;
    }
    struct Node *temp = head;
    for(int i=1;i<pos-1;i++)
        temp = temp->next;
    newnode->next = temp->next;
    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
```

Output

```
10 20 25 30

=== Code Execution Successful ===
```

main.c

```c
    }
    struct Node *temp = head;
    for(int i=1;i<pos-1;i++)
        temp = temp->next;
    newnode->next = temp->next;
    temp->next = newnode;
    return head;
}
void display(struct Node *head)
{
    struct Node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->data);
        temp=temp->next;
    }
}
int main()
{
    struct Node *head=NULL;
    head=insertPos(head,10,1);
    head=insertPos(head,20,2);
    head=insertPos(head,30,3);
    head=insertPos(head,25,3);
    display(head);
}
```

Output

```
10 20 25 30

=== Code Execution Successful ===
```

4.deletion at beginning

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deleteBegin(struct Node *head)
{
    if(head == NULL)
        return NULL;
    struct Node *temp = head;
    head = head->next;
    free(temp);
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

main.c

```c
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1, *n2, *n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
    head = deleteBegin(head);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

5.deletion at end

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deleteEnd(struct Node *head)
{
    if(head == NULL)
    {
        printf("List is empty\n");
        return NULL;
    }
    if(head->next == NULL)
    {
        free(head);
        return NULL;
    }
    struct Node *temp = head;
    while(temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
    return head;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = (struct Node*)malloc(sizeof(struct Node));
    n2 = (struct Node*)malloc(sizeof(struct Node));
    n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
    head = deleteEnd(head);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

6.deletion at given position

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deletePos(struct Node *head, int pos)
{
    struct Node *temp;
    struct Node *del;
    int i
    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }
    if(pos == 1)
    {
        temp = head;
        head = head->next;
        free(temp);
        return head;
    }
    temp = head;
    for(i = 1; i < pos-1; i++)
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```

main.c

```c
    temp = head;
    for(i = 1; i < pos-1; i++)
        temp = temp->next;
    del = temp->next;
    temp->next = del->next;
    free(del);
    return head;
}
void display(struct Node *head)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
```

Memory usage: 596 MB

```
10 30

=== Code Execution Successful ===
```

main.c

```c
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    struct Node *n1,*n2,*n3;
    struct Node *head;
    n1 = malloc(sizeof(struct Node));
    n2 = malloc(sizeof(struct Node));
    n3 = malloc(sizeof(struct Node));
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = NULL;
    printf("Before deletion:\n");
    display(head);
    head = deletePos(head,2);
    printf("\nAfter deletion:\n");
    display(head);
    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 30

=== Code Execution Successful ===
```